

Grocery Buddy - LLM Handoff Brief

1) Project Overview

Grocery Buddy is a React Native mobile app for household grocery planning and shopping. The core idea is to let family members create and manage shared grocery lists, then move into a shopping-focused mode while in-store. The long-term vision is voice-first list management plus real-time collaboration.

2) Product Goal

Build a family grocery app that is:

- ? Fast for list creation and item editing
- ? Easy to use during shopping trips
- ? Designed for future voice commands and collaborative, real-time updates

3) Current Build Status

The app currently has a working local state architecture and core list/item CRUD flows.

Implemented now

- ? Create grocery lists with name + selected store
- ? View active lists on home screen
- ? Open list details by dynamic route
- ? Add items to a specific list
- ? Edit item names inline in list details
- ? Delete items
- ? "Start Shopping" button placeholder in list details
- ? Shared global state via React Context (ListProvider + useLists)

Not fully implemented yet

- ? Dedicated shopping mode experience (currently placeholder alert)
- ? Item completion toggle UI behavior in list details (function exists in context; UI is not active)
- ? Persistent storage (data is in-memory only)
- ? Real-time sync/collaboration backend
- ? Voice command pipeline
- ? In-app chat/messaging

4) Tech Stack

- ? Framework: Expo + React Native
- ? Routing: Expo Router (file-based)
- ? Language: TypeScript
- ? State Management: React Context API + custom hook
- ? UI: React Native components + @expo/vector-icons
- ? Testing setup: Jest (jest-expo, ts-jest, @testing-library/react)

Package highlights

- ? expo ~54.0.31
- ? react 19.1.0
- ? react-native 0.81.5
- ? expo-router ~6.0.21

? react-native-safe-area-context ~5.6.0

5) App Structure

Key folders/files:

- ? app/_layout.tsx: App shell and stack navigation, wraps app in ListProvider
- ? app/index.tsx: Home screen (lists, search UI, create-list CTA)
- ? app/create-list/index.tsx: Create list screen with store picker
- ? app/list/[id].tsx: List details (add/edit/delete items, start-shopping CTA)
- ? lib/state/listContext.tsx: Global list + item state and CRUD operations
- ? lib/types.ts: Type models (GroceryList, GroceryItem, Store)
- ? constants/stores.ts: Store catalog used by create-list

6) Data Models

GroceryList

- ? id: string
- ? name: string
- ? store: string
- ? items: GroceryItem[]
- ? createdAt: string
- ? updatedAt: string

GroceryItem

- ? id: string
- ? name: string
- ? quantity?: string
- ? completed: boolean
- ? createdAt: string
- ? price?: number
- ? addedBy?: string

Store

- ? id: string
- ? name: string
- ? icon: string (emoji)

7) State & Logic (Context API)

lib/state/listContext.tsx provides:

- ? lists
- ? addList(name, store)
- ? updateList(id, updates)
- ? deleteList(id)
- ? getListById(id)
- ? addItemToList(listId, name, quantity?)
- ? updateItem(listId, itemId, updates)
- ? deleteItem(listId, itemId)
- ? toggleItem(listId, itemId)

Notes:

- ? IDs are generated with timestamp + random suffix

- ? State is currently ephemeral (resets on app restart)
- ? Some functions use safe, immutable updates with `setLists(prev => ...)`

8) Current User Flows

Home -> Create list -> List details

1. User opens home screen and taps "Create New List"
2. User enters list name and selects store
3. App creates list in context and navigates to `/list/[id]`
4. User adds items, edits item text inline, or deletes items

List details behavior

- ? Edit icon switches row into inline editing mode
- ? Save button commits update via `updateItem`
- ? Delete icon removes item via `deleteItem`
- ? "Start Shopping" currently triggers placeholder alert

9) Planned Features Roadmap

Near-term

- ? Implement full shopping mode screen/state
- ? Re-enable and polish item completion toggles in UI
- ? Add list-not-found guard and improved empty/error states
- ? Persist data locally (`AsyncStorage` or similar)

Mid-term

- ? Group items by aisle/category during shopping mode
- ? Progress indicator for checked items
- ? Better list search/filtering on home screen

Long-term vision

- ? Voice commands for adding/editing items
- ? Shared family lists with real-time sync
- ? Cloud backend (planned AWS direction):
 - AppSync / GraphQL API
 - Lambda
 - DynamoDB
 - Cognito auth
 - S3 (if attachments/media)
 - Push notifications
- ? In-app messaging/chat tied to a list

10) Constraints / Known Gaps

- ? No backend integration yet
- ? No offline sync strategy yet
- ? No conflict resolution for multi-user edits (future)
- ? Some legacy instructional markdown files exist from early learning scaffolding

11) Suggested Next Tasks for a New LLM

1. Implement true shopping mode route + UI state model.
2. Add persistent local storage and hydration in `ListProvider`.

3. Add robust list/item validation + edge-case handling.
4. Clean unused imports/components and improve consistency in comments.
5. Add tests for context CRUD and list-detail interactions.

12) Quick Prompt You Can Reuse

"You are helping on Grocery Buddy, an Expo React Native + TypeScript app using Expo Router and React Context for state. Current features: list creation, list details, item add/edit/delete, store selection. I need you to continue implementation of shopping mode, persistence, and eventually collaborative/voice features. Please preserve existing architecture unless a clear improvement is necessary, and explain changes with file-level rationale."